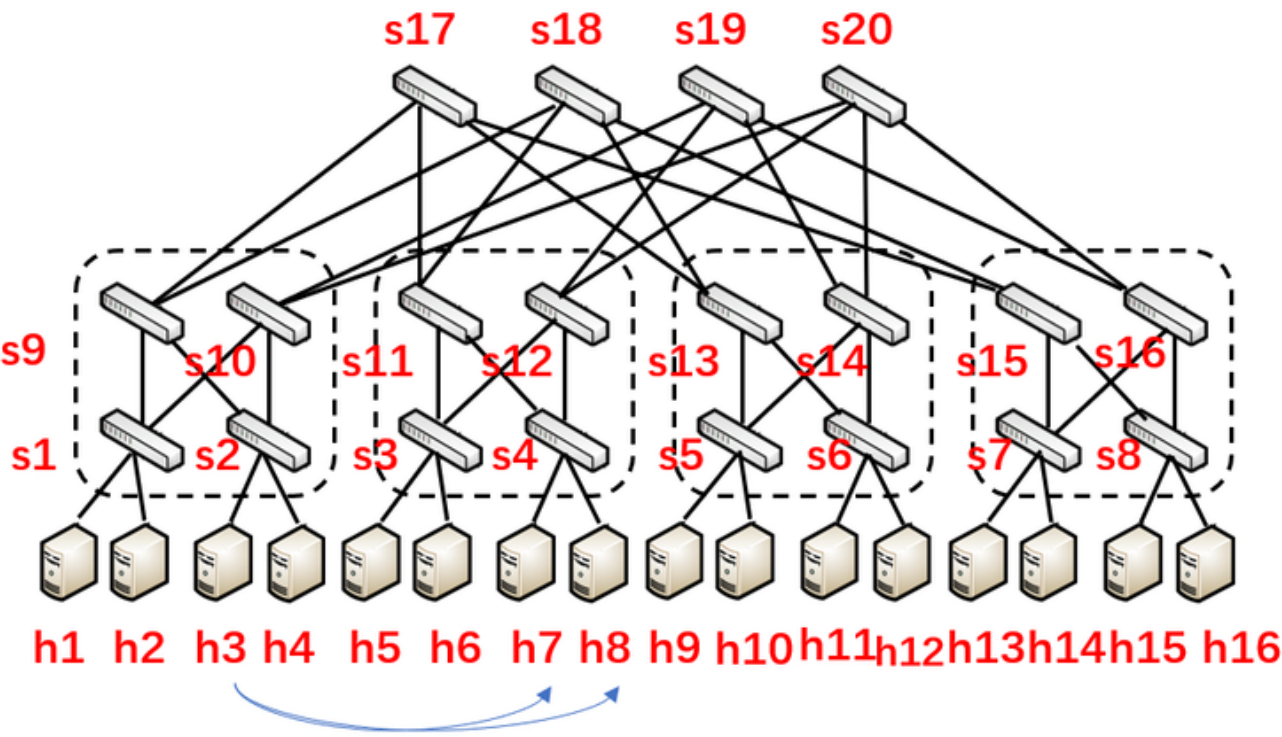


计算机网络H 实验7 实验报告

姓名： 邱深凌

学号： 24300240123

实验任务



基于上述拓扑，实现LPR RSR LLR

代码解释

LPR

Ryu周期性发送LLDP探测链路，触发EventSwitchEnter、EventLinkAdd，这时把链路更新到adjacency即可

在交换机首次连接时_switch_features_handler下发一条table-miss流表匹配所有报文，交换机中没有其他表项时所有未知流量都会PacketIn，再通过_packet_in_handler去下发转发规则

LPR每次选择最靠左的路径，因此先枚举所有从主机到目的主机的路径用min(paths, key=lambda p: tuple(p)) 对路径的交换机ID排序选最小，即可得到这一层的结点

对应的核心代码可以分成三步：先枚举等价最短路径，再按字典序选出最左路径，最后沿着这条路径在每台交换机上安装流表

第一步是根据FatTree结构枚举所有候选路径：

```

src_edge = self.hosts[src_ip][0]
dst_edge = self.hosts[dst_ip][0]

def pod_of_edge(e):
    return (e - 1) // 2

def aggs_in_pod(pod):
    return [9 + 2 * pod, 10 + 2 * pod]

def cores_for_agg(agg):
    return [17, 18] if (agg % 2 == 1) else [19, 20]

def dst_agg_for_core(dst_pod, core):
    return (9 + 2 * dst_pod) if core in [17, 18] else (10 + 2 * dst_pod)

paths = []
if src_edge == dst_edge:
    paths = [[src_edge]]
else:
    sp = pod_of_edge(src_edge)
    dp = pod_of_edge(dst_edge)
    if sp == dp:
        for agg in aggs_in_pod(sp):
            paths.append([src_edge, agg, dst_edge])
    else:
        for agg in aggs_in_pod(sp):
            for core in cores_for_agg(agg):
                paths.append([src_edge, agg, core,
                              dst_agg_for_core(dp, core), dst_edge])

```

通过`pod_of_edge`、`aggs_in_pod`、`cores_for_agg`把从源主机到目的主机的所有等价最短路径枚举出来，，因为一共最多5跳所以直接暴力

第二步是在这些候选路径里选出最左（可达的前提下）：

```

path = min(paths, key=lambda p: tuple(p))

ok = True
for i in range(len(path) - 1):
    if path[i + 1] not in self.adjacency[path[i]]:
        ok = False
        break
if not ok:
    return

```

`min(paths, key=tuple)`即实现检查；随后逐跳检查`adjacency`，如果某一条边还没有在拓扑发现里出现，就丢弃

第三步是在这条路径上的每一台交换机安装这一条转发规则

```

for idx, sw in enumerate(path):
    if sw not in self.datapath_list:
        continue
    dp = self.datapath_list[sw]
    p = dp.ofproto_parser

    if sw == in_dpid:
        this_in = in_port
    elif idx == 0:
        this_in = self.hosts[src_ip][1]
    else:
        prev_sw = path[idx - 1]
        if prev_sw not in self.adjacency[sw]:
            continue
        this_in = self.adjacency[sw][prev_sw]

    if idx == len(path) - 1:
        out_port = self.hosts[dst_ip][1]
    else:
        next_sw = path[idx + 1]
        out_port = self.adjacency[sw][next_sw]

    actions = [p.OFPAActionOutput(out_port)]

    if flow_type == 'ipv4':
        match = p.OFPMatch(eth_type=ether_types.ETH_TYPE_IP,
                           in_port=this_in,
                           ipv4_src=src_ip, ipv4_dst=dst_ip)
        self.add_flow(dp, 100, match, actions)
    else:
        match = p.OFPMatch(eth_type=ether_types.ETH_TYPE_ARP,
                           in_port=this_in,
                           arp_spa=src_ip, arp_tpa=dst_ip)
        self.add_flow(dp, 90, match, actions)

```

对每个交换机，根据其在路径中的位置计算入端口和出端口，如果是PacketIn产生的交换机，则入端口直接使用PacketIn中的`in_port`；如果是靠近主机的一端，则入端口是主机接入该交换机的端口；否则入端口来自上一跳交换机在`adjacency`中记录的端口；出端口如果是路径末尾，则连接到目的主机，否则连接到路径中的下一跳

ARP报文为`eth_type=ARP + in_port + arp_spa + arp_tpa`，保证同一源目对的后续包都能直接被转发，不PacketIn

值得一提的是，如果把入端口和第一跳输出端口写成下面这样，就会产生非常难以理解的报错：

```

if idx == 0:
    this_in = in_port
else:
    prev_sw = path[idx - 1]
    this_in = self.adjacency[sw][prev_sw]

```

```

.....
if len(path) == 1:
    first_out = self.hosts[dst_ip][1]
else:
    first_out = self.adjacency[in_dpid][path[1]]

```

实际上，默认PacketIn一定发生在路径的第一个交换机上会产生问题，报错KeyError

RSR

把排序改为`random.choice(paths)`，收敛（逐跳确认`adjacency[u][v]`存在）之后，随机选中的路径被用于后续下发

和LPR一样先枚举，选择部分修改一下

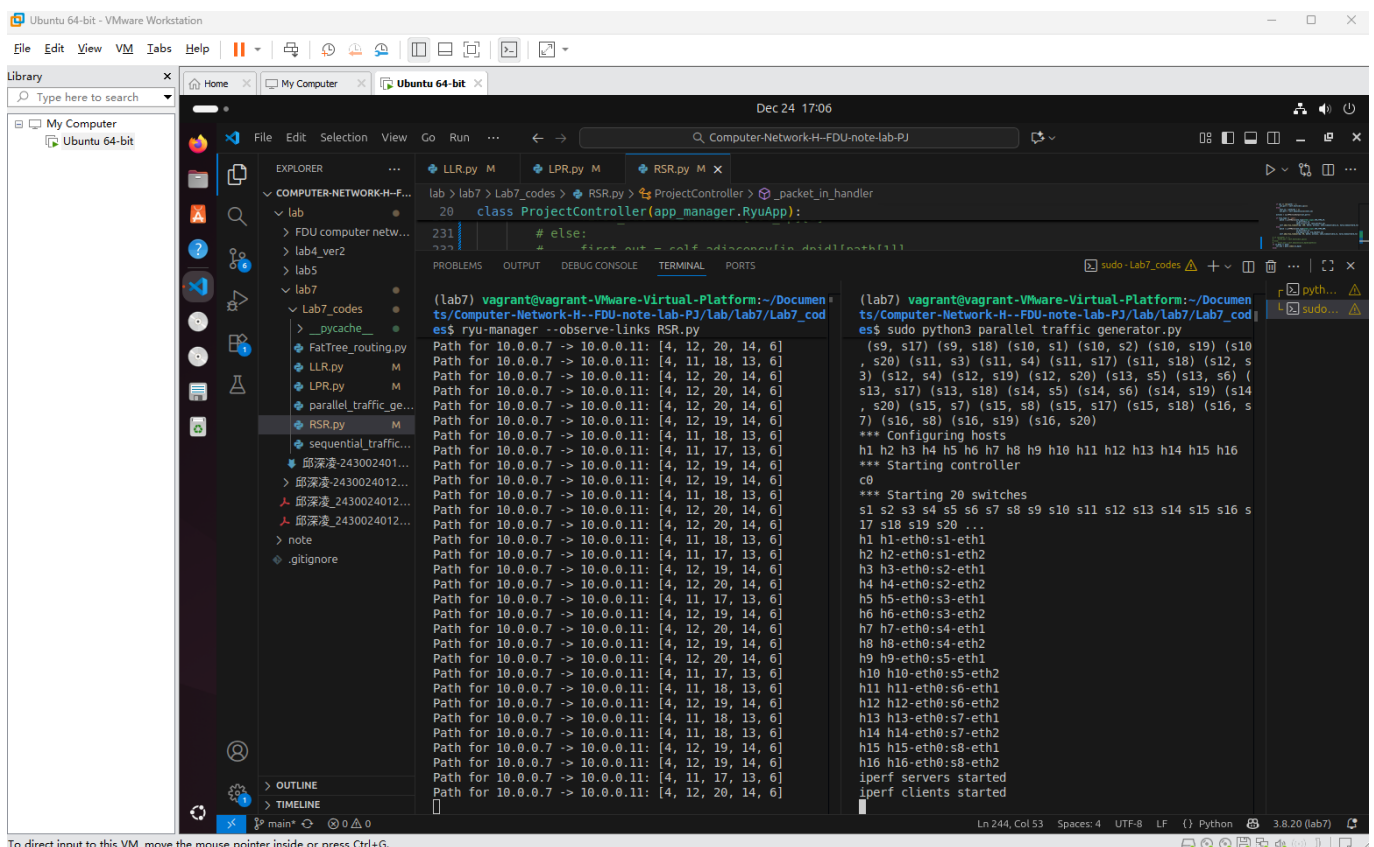
```

path = random.choice(paths)

ok = True
for i in range(len(path) - 1):
    if path[i + 1] not in self.adjacency[path[i]]:
        ok = False
        break
if not ok:
    return

```

`random.choice(paths)`实现了PacketIn，之后逐跳检查每一条链路都已经出现在`adjacency`，之后的安装流表同理



值得一提的是，最开始我尝试加上超时（修改`add_flow`，加上计时器），发现如果不加上`self.debug_printed = set()`，就会出现如上图的问题，打印出无数的路径，永远无法收敛（这也是显然的），没有办法正常实验，所以最后还是去掉了

LLR

LLR在路径选择阶段引入负载，选择当前最空闲的路径

因此，要加上初始化`link_load`，即以`(u, v)` 为key的字典，记录流数量

`flow_chosen`标记已经分配过流量的`(src_ip, dst_ip)`

在候选路径枚举完成并收敛后，对每一条路径计算代价

具体而言，遍历路径上所有相邻交换机`(u, v)`，取其`link_load`的最大值作为瓶颈链路负载，之后在所有路径中选取代价最小的路径（pair的第二项为字典序保证先LLR之后LPR）

对于新流`(src_ip, dst_ip)`，确定路径后沿着这条路径上所有的链路将`link_load`增加1，从而实现流的数量更新

具体实现上，第一段依然是候选路径枚举和收敛检查，同上

第二段是定义路径代价并选出最大链路负载最小的路径，实现如下

```
def path_cost(pth):
    if len(pth) <= 1:
        return 0.0
    mx = 0.0
    for i in range(len(pth) - 1):
        u, v = pth[i], pth[i + 1]
        mx = max(mx, self.link_load[(u, v)])
    return mx

path = min(valid_paths, key=lambda pth: (path_cost(pth), tuple(pth)))
```

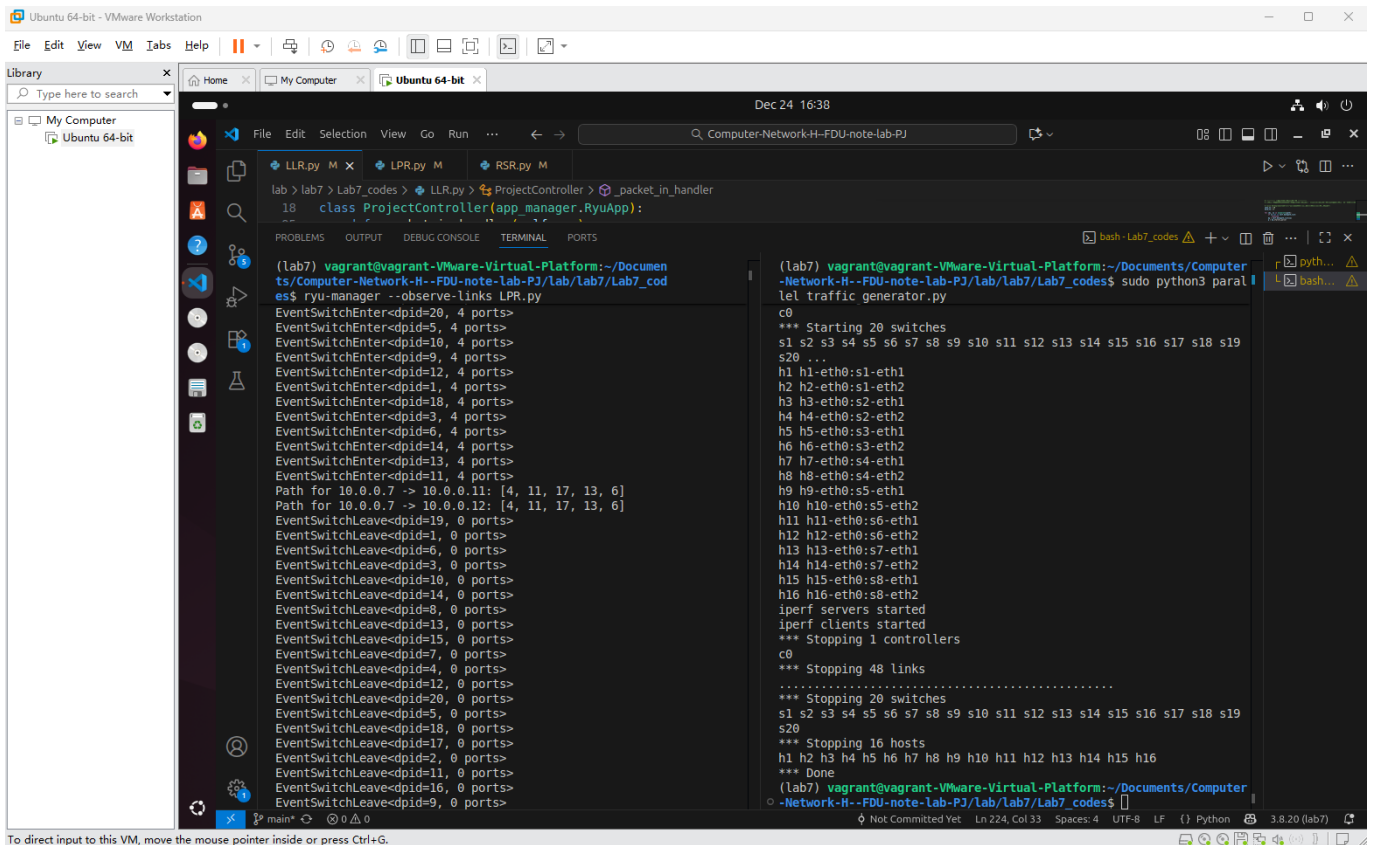
`path_cost`取的是路径上所有链路`link_load`的最大值，即瓶颈链路负载；`min`函数的第一关键字是`path_cost`，第二关键字实现LPR

最后沿路径为每个交换机安装转发规则也同理

实验结果

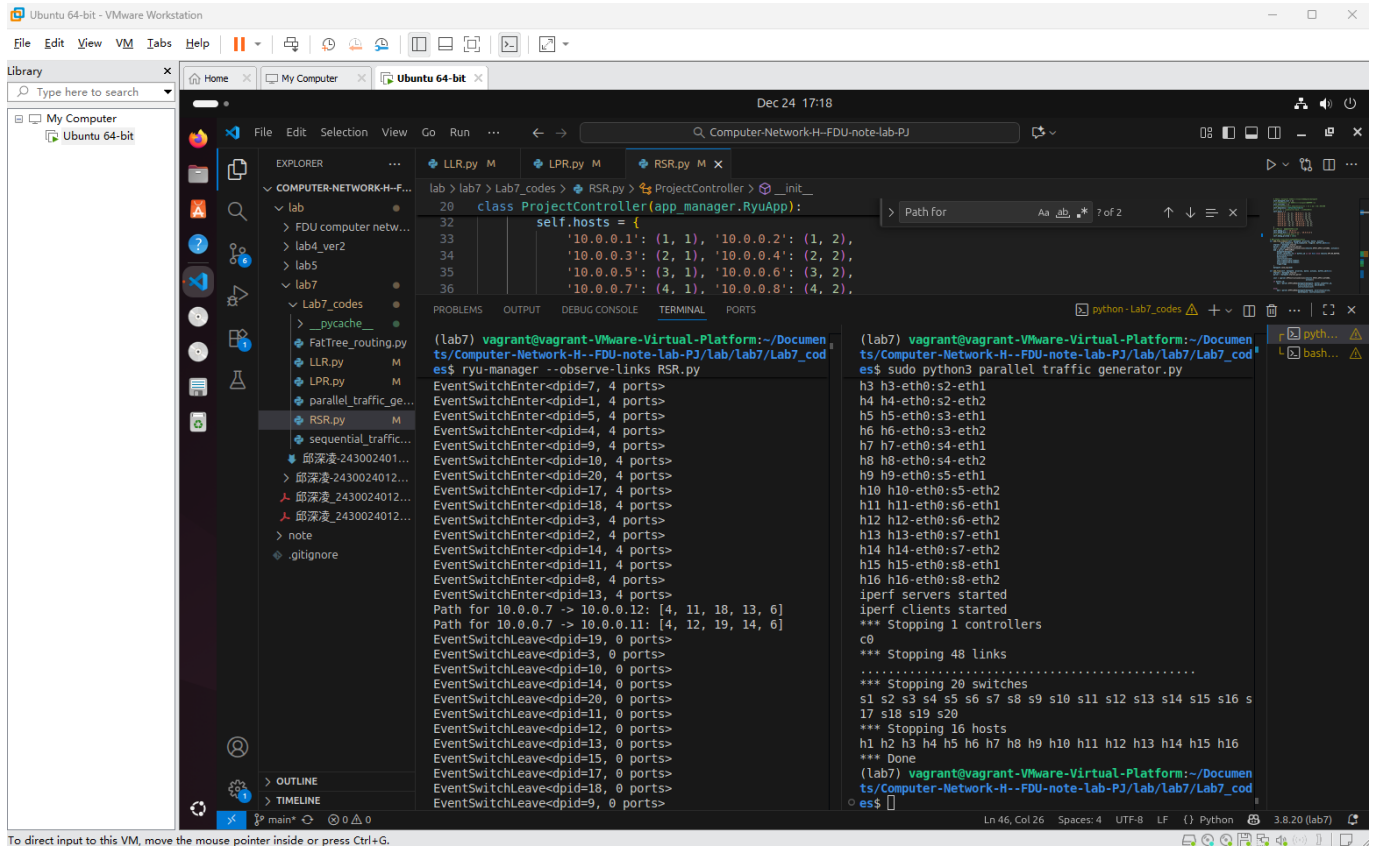
实验过程，参考`demo.mp4`；实验所有输出见`result.txt`

LPR



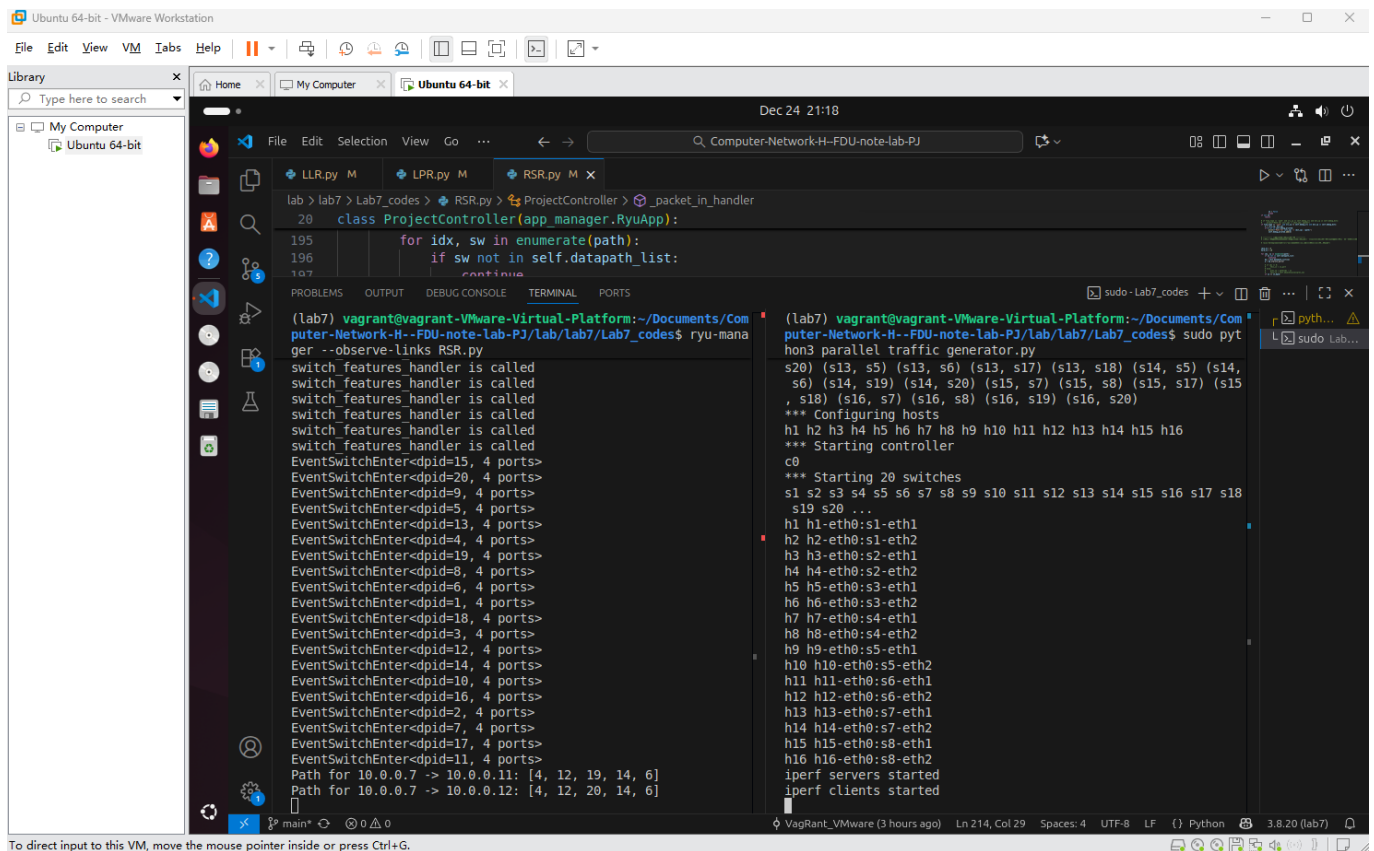
另一方面，由于LPR始终偏向某一侧的核心和汇聚链路，跨pod的流大量集中在相同的交换机上，Event也大致可见这个趋势

6 / 9



```
lab > lab7 > Lab7_codes > RSR.py > ProjectController > _init_
32 class ProjectController(app_manager.RyuApp):
33     self.hosts = {
34         '10.0.0.1': (1, 1), '10.0.0.2': (1, 2),
35         '10.0.0.3': (2, 1), '10.0.0.4': (2, 2),
36         '10.0.0.5': (3, 1), '10.0.0.6': (3, 2),
37         '10.0.0.7': (4, 1), '10.0.0.8': (4, 2),
38     }
39
40     def __call__(self, req):
41         if req.jsonrpc == 'call':
42             if req.method == 'observe_links':
43                 self.observe_links()
44             else:
45                 return {'error': 'Invalid method'}
46         return {'result': 'Success'}
47
48     def observe_links(self):
49         # Get the list of hosts and their connections
50         hosts = self.hosts
51         # Get the list of switch features and their connections
52         switch_features = self.switch_features
53         # Get the list of paths for the hosts and switch features
54         paths = self.paths
55
56         # Print the results
57         print('Hosts:')
58         for host in hosts:
59             print(host)
60
61         print('Switch features:')
62         for switch_feature in switch_features:
63             print(switch_feature)
64
65         print('Paths:')
66         for path in paths:
67             print(path)
68
69         # Stop the traffic generator
70         self.stop_traffic_generator()
71
72         # Done
73         return {'result': 'Success'}
```

RSR中，新流第一次触发PacketIn时随机选择，10.0.0.7 -> 10.0.0.12分别 [4, 11, 18, 13, 6]、[4, 12, 19, 14, 6]



```
lab > lab7 > Lab7_codes > RSR.py > ProjectController > _packet_in_handler
20 class ProjectController(app_manager.RyuApp):
195     for idx, sw in enumerate(path):
196         if sw not in self.datapath_list:
197             continue
198
199     # Print the results
200     print('Hosts:')
201     for host in hosts:
202         print(host)
203
204     print('Switch features:')
205     for switch_feature in switch_features:
206         print(switch_feature)
207
208     print('Paths:')
209     for path in paths:
210         print(path)
211
212     # Stop the traffic generator
213     self.stop_traffic_generator()
214
215     # Done
216     return {'result': 'Success'}
```

第二次运行，得到了不同的链路，也证明了其随机性

实际上相比LPR，RSR更有利于利用所有等价路径，有助于缓解拥塞，当然本实验并未考虑要缓解拥塞

The screenshot shows a VMware Workstation interface with a Linux VM named 'Ubuntu 64-bit'. The VM is running, and the terminal window displays the output of a Python script. The script defines a ProjectController class and instantiates it. The terminal output shows the class being loaded and the controller being instantiated. The interface includes a menu bar, a toolbar, and a sidebar with a file explorer and a terminal window.

File Explorer:

- COMPUTER-NETWORK-H-FDU-note-lab-P3
 - lab
 - FDU computer network...
 - lab2_ver2
 - lab5
 - lab7
 - Lab7_codes
 - __pycache__
 - FatTree_routing.py
 - LLR.py
 - LPR.py
 - parallel_traffic_ge...
 - RSR.py
 - sequential_traffic...
 - 邱深凌_2430024012...
 - 邱深凌_2430024012...
 - 邱深凌_2430024012...
 - note
 - .gitignore

Terminal Output:

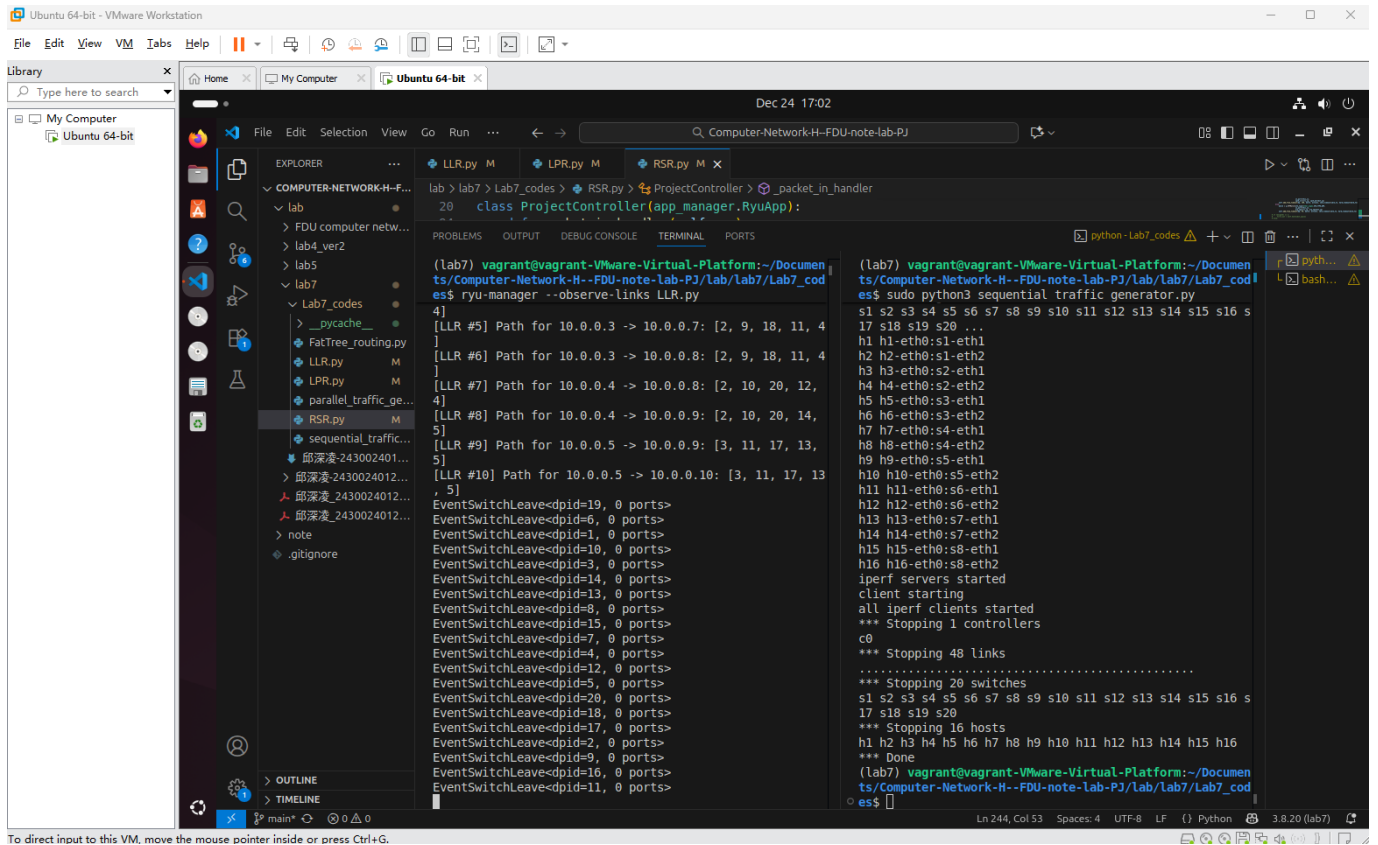
```

(lab7) vagrant@vagrant-Virtual-Platform:~/Documents/Computer-Network-H-FDU-note-lab-P3/Lab7/Lab7_cod
es$ python3 sequential_traffic_generator.py
*** Creating network
*** Adding controller
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
*** Adding switches:
s1 s2 s3 s4 s5 s6 s7 s8 s9 s10 s11 s12 s13 s14 s15 s16 s
17 s18 s19 s20
*** Adding links:
(h1, s1) (h2, s1) (h3, s2) (h4, s2) (h5, s3) (h6, s3) (h
7, s4) (h8, s4) (h9, s5) (h10, s5) (h11, s6) (h12, s6) (
h13, s7) (h14, s7) (h15, s8) (h16, s8) (s9, s1) (s9, s2)
(s9, s17) (s9, s18) (s10, s1) (s10, s2) (s10, s19) (s10
, s20) (s11, s3) (s11, s4) (s11, s17) (s11, s18) (s12, s
3) (s12, s4) (s12, s19) (s12, s20) (s13, s5) (s13, s6) (
s13, s17) (s13, s18) (s13, s5) (s14, s6) (s14, s19) (s14
,s20) (s15, s7) (s15, s8) (s15, s17) (s15, s18) (s16, s
7) (s16, s8) (s16, s19) (s16, s20)
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
*** Starting controller
c0
*** Starting 20 switches
s1 s2 s3 s4 s5 s6 s7 s8 s9 s10 s11 s12 s13 s14 s15 s16 s
17 s18 s19 s20 ...
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s2-eth1
h4 h4-eth0:s2-eth2
h5 h5-eth0:s3-eth1
h6 h6-eth0:s3-eth2
h7 h7-eth0:s4-eth1
h8 h8-eth0:s4-eth2
h9 h9-eth0:s5-eth1
h10 h10-eth0:s5-eth2
  
```

The screenshot shows a VMware Workstation interface with a Ubuntu 64-bit virtual machine. The terminal window displays the execution of a Python script named 'ProjectController.py' which configures a network topology. The script defines 18 links (LLR #1 to LLR #18) and 18 switches (s1 to s18) with their respective IP addresses and port configurations. The output shows the successful creation of the network topology and the starting of IPERF servers.

```

(lab7) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/Computer-Network-H--FDDU-note-Lab-P3/Lab/Lab7/Lab7_cod
es$ python3 sequential_traffic_generator.py
s1 s2 s3 s4 s5 s6 s7 s8 s9 s10 s11 s12 s13 s14 s15 s16 s
17 s18 s19 s20 ...
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s2-eth1
h4 h4-eth0:s2-eth2
h5 h5-eth0:s3-eth1
h6 h6-eth0:s3-eth2
h7 h7-eth0:s4-eth1
h8 h8-eth0:s4-eth2
h9 h9-eth0:s5-eth1
h10 h10-eth0:s5-eth2
h11 h11-eth0:s6-eth1
h12 h12-eth0:s6-eth2
h13 h13-eth0:s7-eth1
h14 h14-eth0:s7-eth2
h15 h15-eth0:s8-eth1
h16 h16-eth0:s8-eth2
iperf servers started
client starting
all iperf clients started
*** Stopping 1 controllers
c0
*** Stopping 48 links
.....
*** Stopping 20 switches
s1 s2 s3 s4 s5 s6 s7 s8 s9 s10 s11 s12 s13 s14 s15 s16 s
17 s18 s19 s20
*** Stopping 16 hosts
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
*** Done
(lab7) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/Computer-Network-H--FDDU-note-Lab-P3/Lab/Lab7/Lab7_cod
es$
  
```

LLR实验使用sequential_traffic_genera主机.py顺序地产生多对主机之间的流量，可以看到，10.0.0.1 -> 10.0.0.5选择了[1, 9, 17, 11, 3]，10.0.0.1 -> 10.0.0.6则选择[1, 10, 19, 12, 3]，两条流分散开来；后续的10.0.0.3 -> 10.0.0.7、10.0.0.3 -> 10.0.0.8、10.0.0.4 -> 10.0.0.8等流也交替使用了17/19、11/12、13/14等不同交换机

随着link_load中流数量的增加，新产生的流导向空闲链路，实现流量均衡分布

附录

参考

<https://zhuanlan.zhihu.com/p/337210249> https://ryu.readthedocs.io/en/latest/ofproto_v1_3_ref.html
<https://github.com/pranav93y/mininet-topology-simulation> <https://github.com/YanHaoChen/Learning-SDN/blob/master/Switch/OpenvSwitch/Walkthrough/README.md>
<https://www.nowcoder.com/discuss/353147667515645952>
https://blog.csdn.net/weixin_45236003/article/details/129745714